

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСИС»**

ИНСТИТУТ Информационных компьютерных наук (ИКН)

КАФЕДРА Автоматизированных систем управления (АСУ)

НАПРАВЛЕНИЕ 09.03.01 ГРУППА БИВТ-24-4

КУРСОВОЙ ПРОЕКТ

ПО КУРСУ Клиент-серверные приложения.

ТЕМА: Разработка клиент-серверного приложения «Учет деревьев в парке»

Студент: Якупова Н.Р.

Проверил: Абросимов Н. А., Шелудяков П. А., Микитенко И.И.

Оценка выполнения курсового проекта _____

Москва 2026 г.

Оглавление

1. Предметная область.....	4
2. Постановка задачи.....	5
2.1 Функциональные требования.....	5
2.2 Требования к технологиям.....	6
3. Описание архитектуры.....	8
3.1 Уровень представления.....	9
3.2 Уровень бизнес-логики.....	9
3.3 Уровень хранения данных.....	10
4. Описание структуры источника данных.....	11
4.1 Пользователи.....	11
4.2 Справочники пород и зон.....	12
4.3 Реестр деревьев.....	12
4.4 Журнал осмотров.....	13
5. Описание серверной части.....	15
5.1 Контроллеры и REST API.....	15
5.2 Сервисы и провайдеры.....	17
5.3 DTO-модели и валидация.....	17
5.4 Аутентификация и управление доступом.....	18
5.5 Обработка ошибок и документация API.....	19
6. Описание клиентской части.....	20
6.1 Страница входа.....	20
6.2 Главная страница.....	21
6.3 Каталог деревьев.....	22
6.4 Профиль сотрудника.....	23
6.5 Справочники и журнал осмотров.....	24
6.6 LocalStorage и клиентская работа с API.....	26

7. Тестирование приложения	27
Заключение.....	29
Список литературы.....	30

1. Предметная область

Городские парки являются важной частью городской инфраструктуры и выполняют одновременно экологическую, рекреационную и эстетическую функции. На территории парка располагается множество деревьев разного возраста, состояния и породного состава. Для администрации парка важно не только учитывать количество деревьев, но и контролировать их текущее состояние, планировать сезонные осмотры и быстро находить проблемные объекты. В противном случае возрастает риск аварийных ситуаций, связанных с сухими ветвями, болезнями растений или несвоевременным уходом [1].

На практике сведения о зеленых насаждениях нередко хранятся разрозненно: часть данных фиксируется в электронных таблицах, часть — в бумажных журналах, а результаты осмотров оформляются в свободной форме. При таком подходе затрудняются поиск нужной записи, контроль истории изменений и анализ текущего состояния насаждений. Кроме того, при ручном ведении учета повышается вероятность ошибок, дублирования информации и потери данных о прошлых инспекциях.

Решением указанных проблем является разработка специализированного клиент-серверного приложения, предназначенного для учета деревьев, хранения справочной информации о породах и зонах парка, а также ведения журнала осмотров. Такая система позволяет централизовать данные, разграничить права доступа между категориями пользователей и обеспечить единый интерфейс для сотрудников парка [6].

В рамках данной курсовой работы рассматривается тема «Дерево», выбранная в виде прикладной задачи учета деревьев в парке. Разрабатываемая информационная система ориентирована на сотрудников парка и должна демонстрировать основные технологии дисциплины.

2. Постановка задачи

Целью курсовой работы является разработка клиент-серверного приложения «Учет деревьев в парке», обеспечивающего централизованное хранение и обработку информации о деревьях, сотрудниках, справочниках и осмотрах. Приложение должно демонстрировать основные возможности изученного стека: React на клиентской стороне, NestJS на серверной стороне, работу с HTTP-запросами, DTO-моделями, валидацией, декораторами, провайдерами и разграничением доступа [1][2].

Исходные записи хранятся в JSON-файле, который считывается при запуске сервера. После инициализации все изменения остаются только в оперативной памяти. Такой подход соответствует индивидуальному согласованию темы и позволяет сделать акцент на клиент-серверной архитектуре, API и бизнес-логике.

2.1 Функциональные требования

Система должна реализовывать обязательные эндпоинты GET /users, GET /users/:id и POST /users, а также прикладной функционал, отражающий тематику проекта. Для пользователей приложения выделены три роли: ADMIN, MANAGER и GUEST. После входа по email и паролю интерфейс должен показывать только разрешенные разделы и действия, соответствующие текущей роли.

Таблица 1 – Функциональные возможности системы

	Реализованные возможности
Обязательные API	Получение списка пользователей, получение

	пользователя по id, создание пользователя
Каталог деревьев	Просмотр карточек деревьев с фотографиями, поиск по названию, породе и зоне
Справочники	Хранение и добавление пород деревьев и зон парка
Осмотры	Ведение журнала инспекций с датой, состоянием и комментарием
Управление доступом	Разделение действий по ролям ADMIN, MANAGER и GUEST
Клиентская логика	Авторизация, маршрутизация, хранение сессии и роли в LocalStorage

Таблица 2 – Права доступа по ролям

Роль	Разрешенные действия
ADMIN	Создание и удаление пользователей, добавление, редактирование и удаление деревьев, управление справочниками, просмотр всех разделов
MANAGER	Просмотр деревьев, проведение осмотров, добавление комментариев, обновление состояния дерева, доступ к журналу осмотров
GUEST	Просмотр каталога деревьев, поиск по каталогу, просмотр общей статистики на главной странице

2.2 Требования к технологиям

- серверная часть должна быть реализована на NestJS и организована по модульному принципу;
- клиентская часть должна быть выполнена на React с использованием HTML, CSS и JavaScript;
- взаимодействие клиента и сервера должно осуществляться по HTTP через REST API;
- в приложении должна быть реализована валидация данных и обработка ошибок;
- доступ к функциям приложения должен зависеть от категории пользователя;
- на клиентской стороне должно использоваться LocalStorage для хранения состояния входа;
- данные должны инициализироваться из JSON-источника и работать в оперативной памяти без сохранения между перезапусками.

В качестве клиентских библиотек в проекте применяются React Router для маршрутизации и Axios для выполнения HTTP-запросов [2][3]. На серверной стороне используются class-validator и class-transformer для проверки DTO-объектов [4]. Для визуальной документации API подключен Swagger [1].

3. Описание архитектуры

Приложение построено по классической трехуровневой клиент-серверной схеме: уровень представления, уровень бизнес-логики и уровень хранения данных. Пользователь взаимодействует с веб-интерфейсом React, который отправляет запросы к API NestJS. Сервер обрабатывает запросы, выполняет бизнес-логику, проверяет права доступа и обращается к централизованному провайдеру хранения данных. Провайдер, в свою очередь, работает с данными, загруженными из JSON-файла в память при запуске сервера.

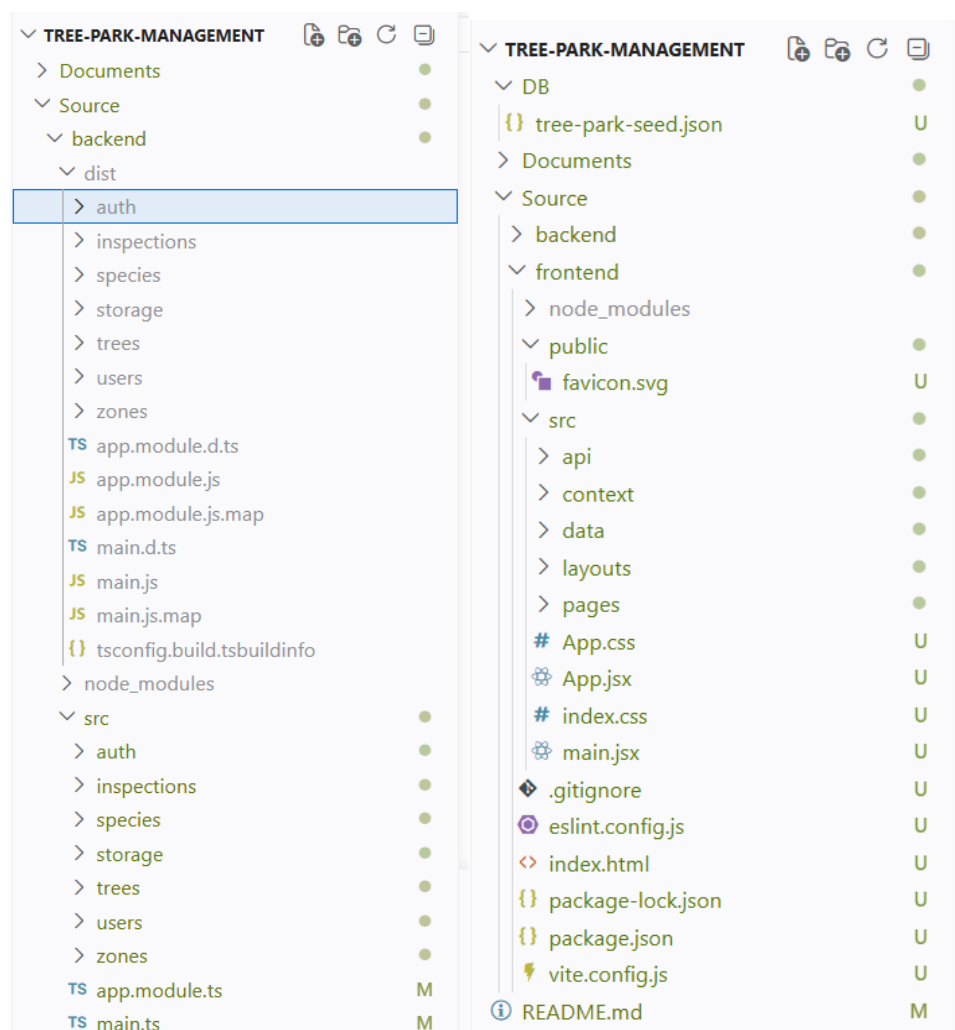


Рисунок 1 – Архитектура приложения Tree Park Management

3.1 Уровень представления

Уровень представления реализован на React 19 и Vite [2][5]. Клиентская часть представляет собой многостраничное одностраничное приложение с маршрутизацией по адресам /login, /, /trees, /profile, /directory и /inspections. Пользователь сначала проходит авторизацию, после чего в зависимости от роли получает доступ только к тем разделам, которые разрешены политикой приложения.

Интерфейс ориентирован на быстрый доступ к прикладным операциям. Для этого на главной странице показываются сводные метрики и последний осмотр, в каталоге деревьев — карточки с фотографиями, а на страницах профиля, справочников и осмотров сосредоточены формы управления данными. Такой подход делает приложение удобным для демонстрации на защите и понятным для конечного пользователя.

3.2 Уровень бизнес-логики

Бизнес-логика реализована на NestJS 11 и организована по модульной структуре: AuthModule, UsersModule, SpeciesModule, ZonesModule, TreesModule, InspectionsModule и StorageModule. Каждый модуль включает контроллер и сервис. Контроллер принимает HTTP-запросы и передает данные в сервисный слой, а сервис выполняет прикладную логику: поиск сущностей, проверку ограничений, создание записей и выдачу связанной информации.

Отдельное место в архитектуре занимает модуль аутентификации. Он обеспечивает вход по email и паролю, а также формирование публичной модели пользователя, которая возвращается в ответе клиенту. Разграничение

доступа выполняется с помощью перечисления ролей, пользовательского декоратора Roles и guard-компонента RolesGuard.

3.3 Уровень хранения данных

Вместо постоянной базы данных в проекте используется JSON-источник данных DB/tree-park-seed.json. При запуске приложения StorageService считывает файл, копирует коллекции в память и затем предоставляет их остальным сервисам. За счет метода getNextId провайдер также отвечает за генерацию новых идентификаторов. Такой вариант хранения не требует внешней СУБД и полностью соответствует условию, по которому данные не должны сохраняться между перезапусками приложения.

Логическая структура данных при этом сохраняет полноценную предметную модель: есть сотрудники, породы деревьев, зоны парка, сами деревья и журнал инспекций. Следовательно, несмотря на отсутствие постоянной БД, проект демонстрирует проектирование структуры данных и работу с взаимосвязанными сущностями.

4. Описание структуры источника данных

Источник данных системы представлен одним JSON-файлом, который содержит пять коллекций: users, species, zones, trees и inspections. Каждая коллекция является аналогом отдельной сущности предметной области. Связи между коллекциями организованы через идентификаторы: tree содержит speciesId и zoneId, а inspection содержит treeId. Таким образом, структура источника данных сохраняет ту же логику, что и традиционная схема базы данных.

Таблица 3 – Коллекции источника данных

Коллекция	Назначение	Связи
users	Сотрудники системы	Независимая коллекция, используется при входе
species	Справочник пород деревьев	Связь один-ко-многим с trees
zones	Справочник зон парка	Связь один-ко-многим с trees
trees	Основной реестр деревьев	Ссылается на species и zones
inspections	Журнал осмотров	Ссылается на trees

4.1 Пользователи

Коллекция users хранит учетные записи сотрудников и используется при авторизации. В проекте предусмотрены три стандартных записи: администратор, сотрудник парка и гостевой пользователь. Для

демонстрационных целей пароль также хранится внутри коллекции. При выдаче данных клиенту поле password скрывается сервисом userService.

Таблица 4 – Поля коллекции users

Поле	Тип	Назначение
id	number	Уникальный идентификатор пользователя
name	string	ФИО сотрудника
email	string	Адрес электронной почты для входа
age	number	Возраст сотрудника, поле необязательно
role	string	Роль доступа: ADMIN, MANAGER или GUEST
password	string	Пароль, используемый при авторизации

4.2 Справочники пород и зон

Коллекции species и zones являются справочными. Первая определяет вид дерева, а вторая задает место размещения на территории парка. Каждая карточка дерева обязательно ссылается на одну породу и одну зону, что позволяет не дублировать одинаковые названия в реестре и упрощает сопровождение данных.

4.3 Реестр деревьев

Коллекция trees является центральной частью системы. Для каждого дерева в ней хранятся название, возраст, высота, дата посадки и ссылки на породу и зону. Эта информация используется как для показа карточек в каталоге, так и для выбора дерева при проведении осмотра. В

пользовательском интерфейсе дерево дополнительно сопровождается изображением, которое подбирается на основе названия породы.

Таблица 6 – Поля коллекции trees

Поле	Тип	Назначение
id	number	Уникальный идентификатор дерева
name	string	Название или условное имя дерева
age	number	Возраст дерева в годах
height	number	Высота дерева в метрах
plantingDate	string	Дата посадки в формате ISO
speciesId	number	Ссылка на запись в коллекции species
zoneId	number	Ссылка на запись в коллекции zones

4.4 Журнал осмотров

Коллекция inspections фиксирует результаты наблюдений за состоянием деревьев. Для каждой записи указываются дата осмотра, состояние, комментарий и идентификатор дерева. Наличие отдельного журнала позволяет отслеживать историю проверок и анализировать, какие объекты требуют дополнительного внимания.

Таблица 7 – Поля коллекции inspections

Поле	Тип	Назначение
------	-----	------------

id	number	Уникальный идентификатор осмотра
inspectionDate	string	Дата проведения осмотра
condition	string	Текстовая оценка состояния дерева
comment	string	Комментарий сотрудника
treeId	number	Ссылка на дерево из коллекции trees

5 Описание серверной части

Серверная часть приложения реализована на TypeScript с использованием NestJS [1]. Контроллеры отвечают за HTTP-маршруты, сервисы инкапсулируют бизнес-логику, а общий провайдер StorageService предоставляет доступ к данным. Такой подход делает проект модульным, расширяемым и удобным для сопровождения.



Рисунок 2 – Структура серверной части приложения

5.1 Контроллеры и REST API

В проекте реализованы шесть контроллеров: AuthController, UsersController, SpeciesController, ZonesController, TreesController и InspectionsController. Они принимают запросы от клиента, используют DTO-

модели в качестве входных параметров и вызывают методы сервисов. Обязательные по условию эндпоинты GET /users, GET /users/:id и POST /users реализованы в полном объеме. Дополнительно приложение содержит маршруты для входа, справочников, деревьев и осмотров.

Таблица 8 – Основные маршруты API

Метод	Маршрут	Назначение
POST	/auth/login	Вход в систему по email и паролю
GET	/users	Получение списка всех пользователей
GET	/users/:id	Получение пользователя по идентификатору
POST	/users	Создание нового пользователя
PUT	/users/:id	Изменение учетной записи
DELETE	/users/:id	Удаление пользователя
GET	/trees	Получение каталога деревьев
POST	/trees	Добавление дерева
PUT	/trees/:id	Редактирование дерева
DELETE	/trees/:id	Удаление дерева
GET	/species	Получение списка пород
POST	/species	Добавление породы
GET	/zones	Получение списка зон

POST	/zones	Добавление зоны
GET	/inspections	Получение журнала осмотров
POST	/inspections	Создание записи осмотра

5.2 Сервисы и провайдеры

Сервисный слой инкапсулирует прикладную логику. Например, `UserService` отвечает за создание и удаление пользователей, проверку уникальности email и формирование публичной модели пользователя без пароля. `TreesService` формирует карточки деревьев со связанными объектами `species` и `zone`. `InspectionsService` связывает результаты осмотров с конкретными деревьями. `Centralized StorageService` выступает провайдером общего доступа к коллекциям и обеспечивает генерацию новых идентификаторов.

Использование провайдера особенно важно для демонстрации возможностей NestJS. Сервисы не знают, из какого физического источника были получены данные, а значит архитектура допускает дальнейшую замену JSON-хранилища на полноценную СУБД без изменения маршрутов и клиентской части.

5.3 DTO-модели и валидация

Для входных данных используются DTO-модели `LoginDto`, `CreateUserDto`, `UpdateUserDto`, `CreateTreeDto`, `UpdateTreeDto`, `CreateSpeciesDto`, `CreateZoneDto` и `CreateInspectionDto`. В этих классах применяются декораторы `class-validator`: `IsEmail`, `IsString`, `IsNotEmpty`,

IsOptional, IsEnum, IsInt, IsNumber, Min, MinLength и IsDateString [4]. В результате сервер заранее отсекает некорректные запросы и возвращает унифицированные сообщения об ошибках.

Листинг 1 – Глобальная настройка ValidationPipe

```
app.useGlobalPipes(  
  new ValidationPipe({  
    whitelist: true,  
    forbidNonWhitelisted: true,  
    transform: true,  
  } ),  
);
```

5.4 Аутентификация и управление доступом

Вход в систему выполняется через POST /auth/login. Пользователь передает email и пароль, после чего AuthService ищет запись по email и сравнивает пароль с данными, загруженными в память. При успешном входе клиент получает объект пользователя и сохраняет его в LocalStorage. Реальная политика доступа определяется ролями ADMIN, MANAGER и GUEST, указанными в перечислении Role.

Для защиты маршрутов применяется собственный декоратор Roles и guard-компонент RolesGuard. Guard читает значение HTTP-заголовка role и сравнивает его с требуемыми ролями маршрута. Если декоратор не указан, маршрут считается открытым. Такой подход позволяет просто и наглядно ограничивать доступ к административным операциям.

Листинг 2 – Проверка ролей в RolesGuard

```
const requiredRoles =  
this.reflector.getAllAndOverride<Role[]>(
```

```
    ROLES_KEY,  
    [context.getHandler(), context.getClass()],  
);  
  
if (!requiredRoles) {  
    return true;  
}  
  
const request = context.switchToHttp().getRequest();  
const role = request.headers['role'];  
return requiredRoles.includes(role as Role);
```

5.5 Обработка ошибок и документация API

Прикладные сервисы используют встроенные исключения NestJS: `NotFoundException`, `ConflictException` и `UnauthorizedException`. Например, если пользователь не найден, сервер возвращает ошибку 404, а если email уже занят — ошибку 409. При неверном пароле возвращается ошибка 401. Для запросов с лишними или неверно типизированными полями `ValidationPipe` генерирует сообщение об ошибке валидации. На стороне клиента эти ошибки отображаются в едином информационном блоке.

Для визуального описания API в приложении настроен Swagger. После запуска сервера документация доступна по адресу `/api`. Это облегчает тестирование и помогает быстро показать структуру маршрутов, DTO-моделей и параметров запросов во время защиты [1].

6 Описание клиентской части

Клиентская часть реализована на React и разбита на маршруты, которые управляются через react-router-dom [2]. Корневой компонент App определяет защищенные и публичные маршруты. Перед открытием внутренних страниц выполняется проверка наличия сессии. Благодаря этому пользователь без входа автоматически перенаправляется на страницу авторизации.

Состояние приложения хранится в контексте ParkContext. Он отвечает за загрузку данных из API, хранение роли и учетной записи, управление формами, а также за вызовы createUser, createTree, updateTree, deleteTree, createInspection и createDirectoryItems. Контекст объединяет все основные действия интерфейса и служит связующим слоем между визуальными компонентами и API.

6.1 Страница входа

Страница входа открывается по маршруту /login. Пользователь вводит email и пароль, после чего приложение отправляет запрос на /auth/login. При успешной проверке сессия сохраняется в LocalStorage, а пользователь перенаправляется на главную страницу. Для демонстрации в интерфейсе предусмотрен блок с тестовыми учетными записями.

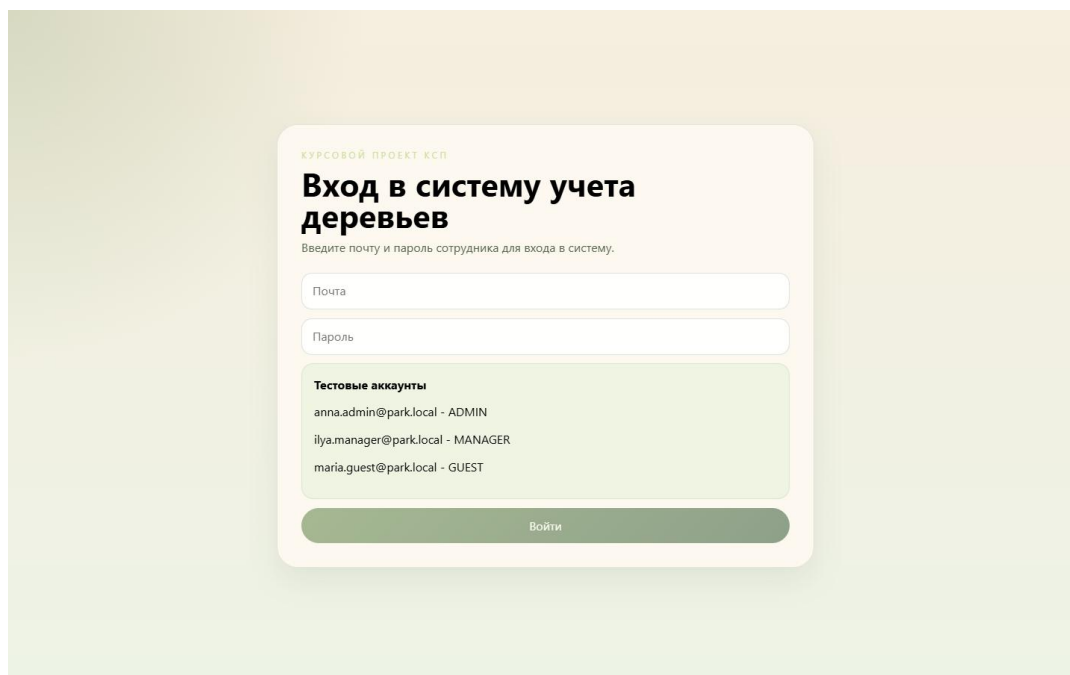


Рисунок 3 – Страница входа в систему

6.2 Главная страница

Главная страница содержит навигацию по разрешенным разделам, карточки сводной статистики и блок с последним осмотром. Набор ссылок в боковом меню зависит от текущей роли. Для администратора доступны все разделы, для сотрудника — каталог деревьев, профиль и осмотры, для гостя — только просмотр каталога и профиля.

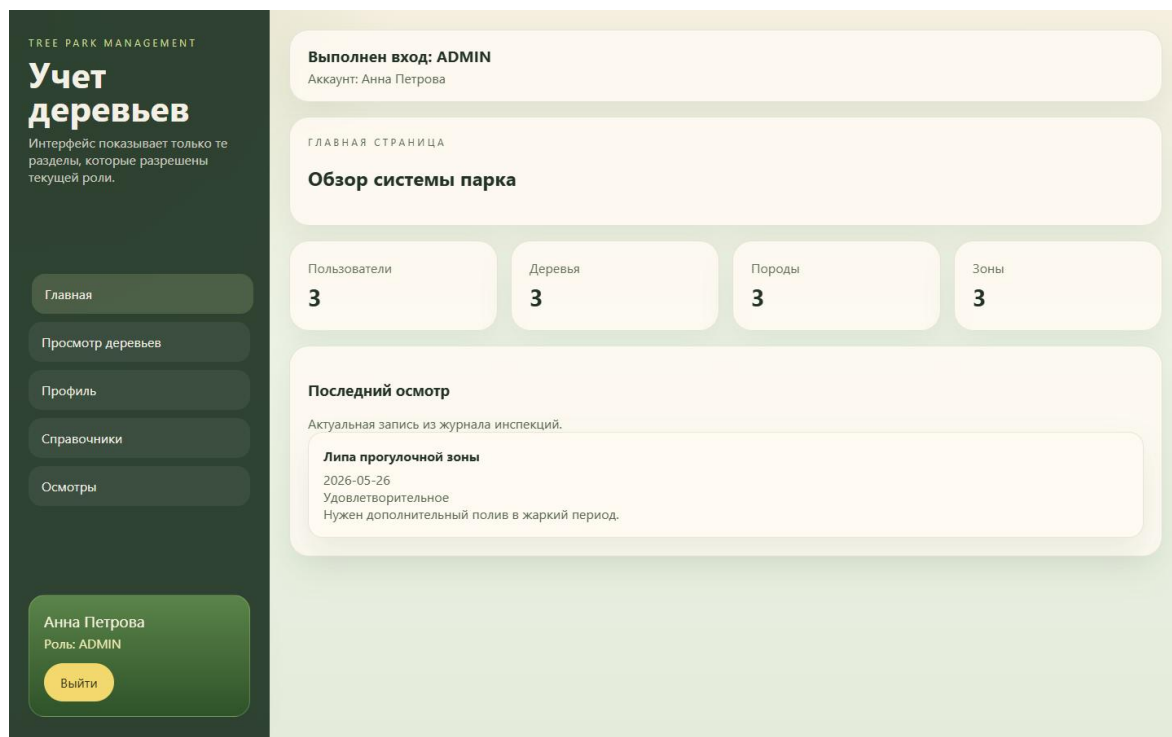


Рисунок 4 – Главная страница приложения

6.3 Каталог деревьев

Страница /trees предназначена для просмотра реестра деревьев. Каждое дерево отображается в виде карточки с фотографией, названием, породой, зоной, возрастом, высотой и датой посадки. Поиск осуществляется по названию дерева, породе и зоне. Для ролей ADMIN и MANAGER доступны формы добавления и редактирования, а для ADMIN также предусмотрено удаление дерева.

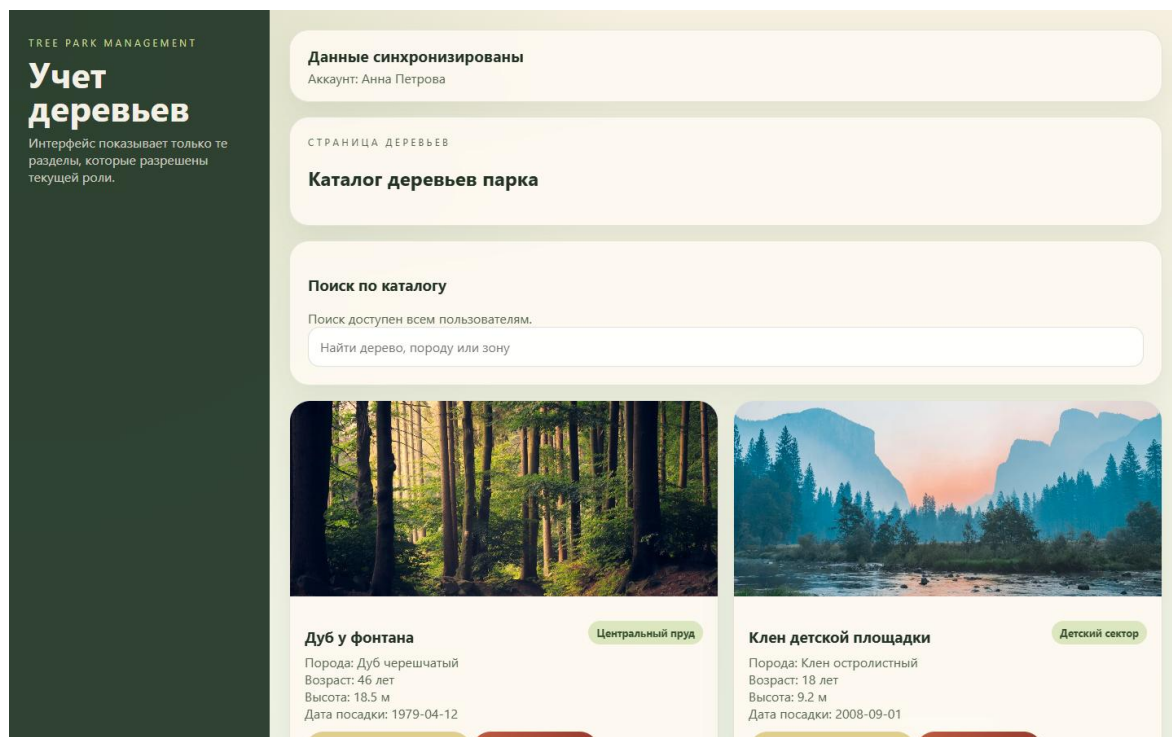


Рисунок 5 – Страница каталога деревьев

6.4 Профиль сотрудника

Страница /profile содержит карточку текущего пользователя, перечень разрешенных действий и, для администратора, интерфейс управления сотрудниками. Здесь можно создать новую учетную запись, указав имя, email, возраст, роль и пароль, а также удалить существующего пользователя. Таким образом, страница профиля объединяет персональную информацию и административные функции.

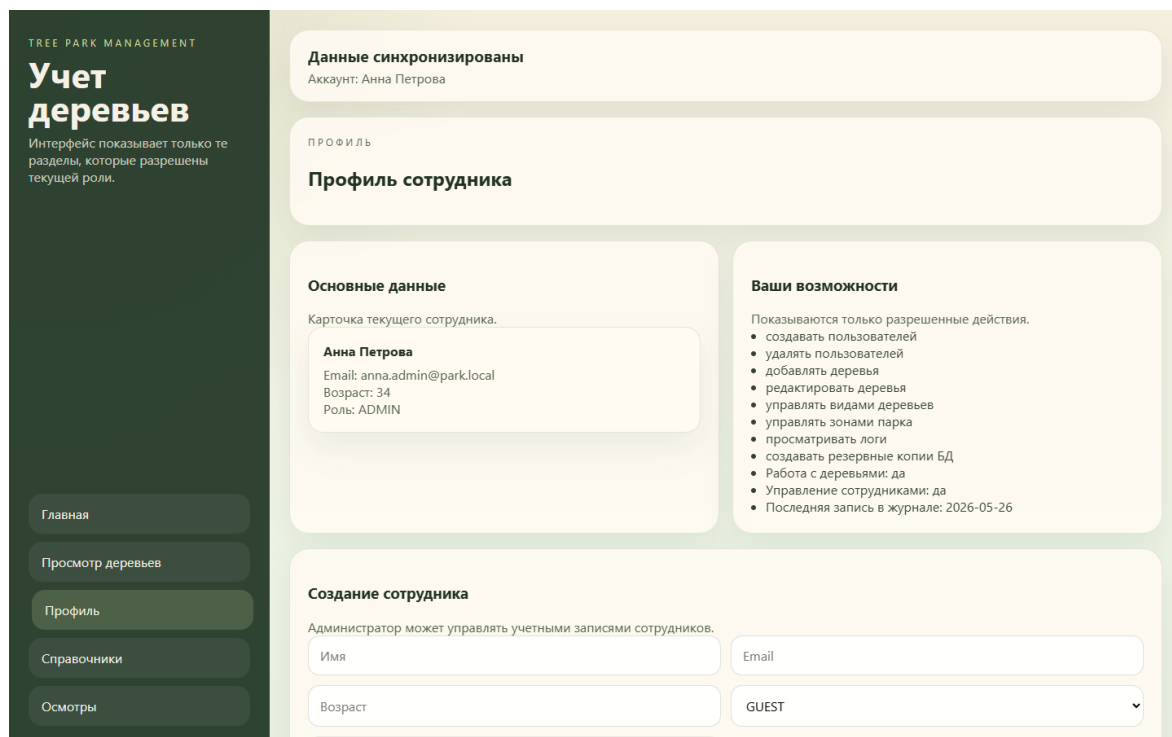


Рисунок 6 – Страница профиля сотрудника

6.5 Справочники и журнал осмотров

Страница /directory предназначена для работы со справочниками пород и зон парка. Она показывает текущие записи и позволяет добавлять новые элементы. Страница /inspections отображает журнал осмотров и форму создания новой записи. Обе страницы связаны с основной логикой учета деревьев: без справочников нельзя создать дерево, а без журнала осмотров невозможно фиксировать изменения состояния объектов.

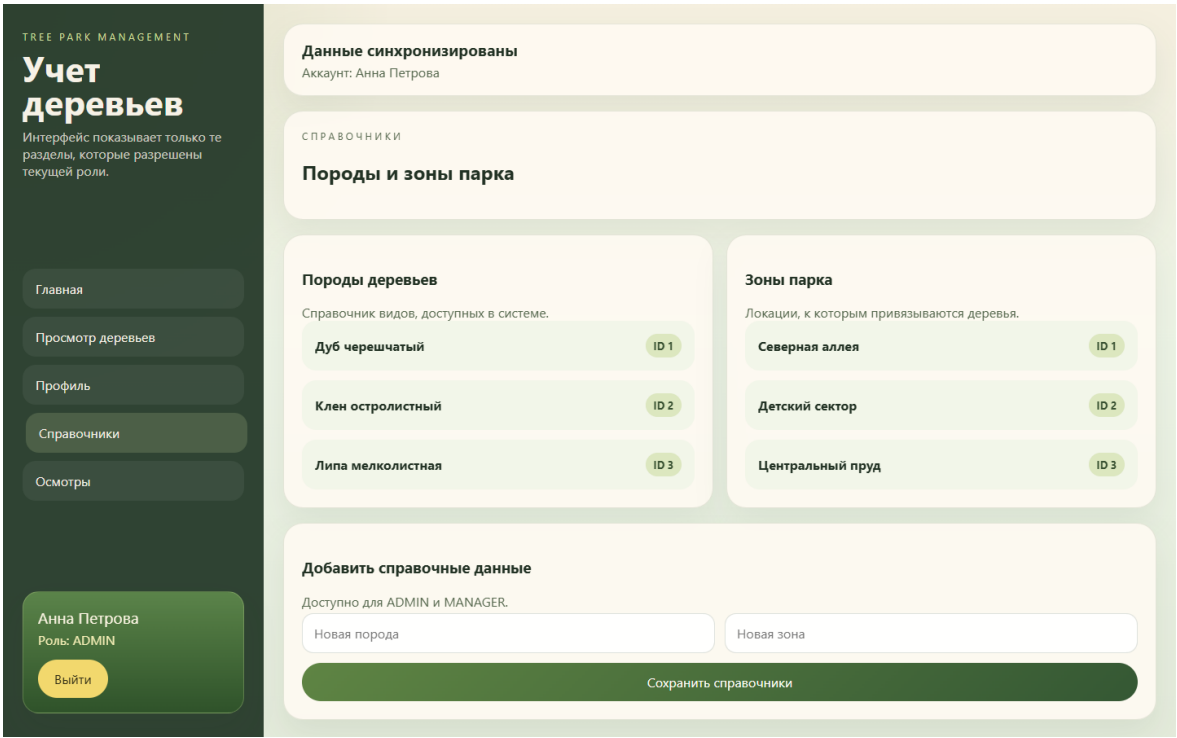


Рисунок 7 – Страница справочников

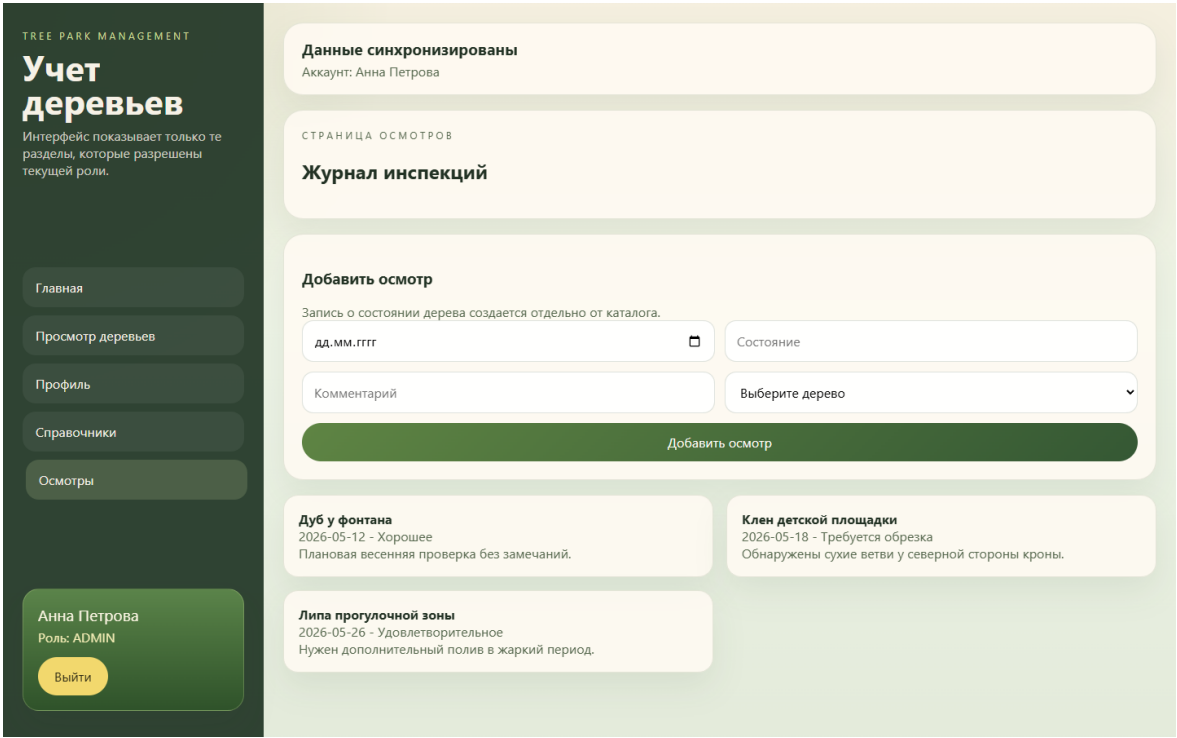


Рисунок 8 – Страница журнала осмотров

6.6 LocalStorage и клиентская работа с API

Для хранения состояния входа используются ключи `tree-park-session` и `tree-park-role`. После входа объект пользователя и роль сохраняются в LocalStorage, а после выхода удаляются. При каждом запросе Axios interceptor добавляет значение роли в заголовок `role`, что позволяет серверу применять RolesGuard. За счет этого логика разграничения доступа синхронизирована между клиентом и сервером.

Фотографии деревьев не хранятся в локальной базе данных, а подбираются функцией `getTreePhoto` на основе названия породы. Для дуба, клена и липы используются заранее заданные ссылки на изображения Unsplash. Благодаря этому каталог деревьев выглядит наглядно и ближе к реальной прикладной системе [8].

7 Тестирование приложения

Проверка проекта выполнялась на двух уровнях: сборка исходного кода и функциональное тестирование пользовательских сценариев. Для серверной части использовалась команда `npm run build` в каталоге `Source/backend`, а для клиентской — `npm run build` в каталоге `Source/frontend`. Успешное завершение обеих сборок подтверждает корректность структуры модулей, DTO-моделей, маршрутов React и JSX-разметки.

Листинг 3 – Команды запуска и сборки проекта

```
cd Source/backend
npm install
npm run start

cd Source/frontend
npm install
npm run dev
```

Функциональное тестирование включало вход под разными ролями, просмотр каталога деревьев, создание пользователя администратором, добавление дерева менеджером, создание записи осмотра и проверку запрета доступа для гостя. Дополнительно проверялись сценарии валидации: неверный email, отсутствие обязательных полей и попытка создания дублирующей записи пользователя.

Таблица 9 – Сценарии тестирования

№	Сценарий	Ожидаемый результат	Фактический результат
1	POST /auth/login с	Пользователь входит	Успешно

	корректным email и паролем	в систему	
2	GET /users	Возвращается список пользователей	Успешно
3	GET /users/1	Возвращается пользователь с id=1	Успешно
4	POST /users с некорректным email	Ошибка валидации 400	Успешно
5	POST /users под ролью ADMIN	Создается новый пользователь	Успешно
6	POST /trees под ролью MANAGER	Создается новое дерево	Успешно
7	DELETE /trees под ролью ADMIN	Дерево удаляется из реестра	Успешно
8	POST /users под ролью GUEST	Доступ запрещен	Успешно
9	POST /inspections под ролью MANAGER	Создается запись в журнале осмотров	Успешно

Проведенное тестирование показало, что приложение выполняет все основные функции, предусмотренные техническим заданием. Сервер корректно валидирует входные данные, клиент отображает ошибки в понятном виде, а ограничения ролей применяются как на уровне маршрутов API, так и на уровне пользовательского интерфейса.

Заключение

В ходе выполнения курсовой работы было разработано клиент-серверное приложение «Учет деревьев в парке». Предметной областью проекта является учет зеленых насаждений на территории городского парка. Основная решаемая проблема — отсутствие единого цифрового инструмента для хранения сведений о деревьях, зонах, породах и результатах осмотров.

Для решения поставленной задачи была выбрана архитектура на базе React и NestJS. Клиентская часть реализует страницы входа, главной панели, каталога деревьев, профиля, справочников и журнала осмотров. Серверная часть организована по модульному принципу, использует контроллеры, сервисы, DTO, декораторы и провайдеры. В качестве источника данных применен JSON-файл с загрузкой в оперативную память, что соответствует условию о несохранении данных между перезапусками приложения.

В результате получено работоспособное приложение, демонстрирующее обязательные эндпоинты пользователей, валидацию входных данных, обработку ошибок, разграничение доступа, клиентскую работу с LocalStorage и прикладной функционал по теме учета деревьев. Проект может быть расширен за счет внедрения постоянной базы данных, более сложной аутентификации, экспорта отчетов и дополнительной аналитики по состоянию парка.

Список литературы

1. NestJS Documentation [Электронный ресурс]. URL: <https://docs.nestjs.com/> (дата обращения: 04.06.2026).
2. React Documentation [Электронный ресурс]. URL: <https://react.dev/> (дата обращения: 04.06.2026).
3. Axios Documentation [Электронный ресурс]. URL: <https://axios-http.com/> (дата обращения: 04.06.2026).
4. class-validator Documentation [Электронный ресурс]. URL: <https://github.com/typestack/class-validator> (дата обращения: 04.06.2026).
5. Vite Documentation [Электронный ресурс]. URL: <https://vite.dev/> (дата обращения: 04.06.2026).
6. Требования к курсовой работе по дисциплине «Клиент-серверные приложения». PDF-документ преподавателя.
7. Unsplash [Электронный ресурс]. URL: <https://unsplash.com/> (дата обращения: 04.06.2026).